

Experiment 6: INHERITANCE

Aim: To implement programs to develop programs that demonstrate the use of Inheritance in creating objects and classes.

Problem Definition: Write Java programs to:

BATCH A

Q1. Single Inheritance – Constructor Chaining and **super**

Design a class **Person** that contains the following attributes: **name**, **age**

The class should include:

- A **parameterized constructor** to initialize these attributes.
- A method **displayPersonDetails()**.

Create a subclass **Employee** that extends **Person** and adds:

- **employeeId**
- **salary**

Requirements:

1. Use **constructor chaining** to initialize the inherited attributes.
2. Use the **super keyword** to call the constructor of the base class.
3. Add a method **displayEmployeeDetails()** that displays all details.
4. Write a main program that creates an **Employee** object and demonstrates the constructor chaining process.

Q2. Multilevel Inheritance – Constructor Execution Order

Create a three-level inheritance hierarchy: **Person** → **Staff** → **Professor**

Person

- Attributes: **name**, **age**
- Parameterized constructor

Staff

- Attribute: `staffId`
- Constructor that initializes `staffId`

Professor

- Attributes: `department`, `researchArea`

Requirements:

1. Use **constructor chaining across all three classes**.
2. Display messages inside each constructor to show the **order of execution**.
3. Use the **super keyword** to pass parameters to parent constructors.
4. Write a main program that creates a `Professor` object and displays all information.

Q3. Hierarchical Inheritance – Dynamic Binding

Create a base class `Vehicle` containing:

- `vehicleNumber`
- `ownerName`
- Method `calculateServiceCost()`

Create the following subclasses:

- `Car`
- `Bike`
- `Truck`

Each subclass should implement its own logic for **service cost calculation** based on different parameters such as:

- engine capacity
- number of service hours

- spare part cost

Requirements:

1. Create a **reference variable of type Vehicle**.
2. Assign objects of different subclasses to it.
3. Demonstrate **dynamic binding** by invoking `calculateServiceCost()` using the base class reference.

Q4. Abstract Class – Runtime Polymorphism

Create an **abstract class Shape** with:

- attribute `color`
- constructor to initialize color
- abstract method `computeArea()`
- method `displayColor()`

Create subclasses:

- `Circle`
- `Rectangle`

Requirements:

1. Each subclass should compute the area based on its own attributes.
2. Use **constructor chaining using super**.
3. In the main program, create an **array of Shape references** and store objects of both subclasses.
4. Call `computeArea()` using the base reference to demonstrate **dynamic binding**.

Q5. Final Class – Restricting Inheritance

Create a **final class SecurityManager** responsible for validating system users.

- Attributes: `username`, `password`
- Parameterized constructor
- Method `validateUser()` that checks if the password length is greater than 6 characters.

Requirements:

1. Demonstrate object creation and validation.
Attempt to create another class `AdvancedSecurityManager` that extends `SecurityManager`.
2. Observe and explain the **compiler error generated due to the final class restriction**.

BATCH B

Q1. Single Inheritance – **super** for Constructor and Method Access

Create a class **Product** containing:

- **productId**
- **productName**
- **price**

Include a parameterized constructor and a method **displayProduct()**.

Create a subclass **ElectronicProduct** that adds:

- **warrantyPeriod**
- **brand**

Requirements:

1. Use **super()** to initialize the parent class attributes.
2. Create a method **displayElectronicProduct()** that uses **super.displayProduct()**.
3. Write a main program demonstrating constructor chaining.

Q2. Multilevel Inheritance – Academic System

Design a class hierarchy: **University → Department → Student**

University

- **universityName**
- **location**

Department

- **departmentName**
- **facultyCount**

Student

- **studentId**
- **course**

Requirements:

1. Implement **constructor chaining across all three classes**.
2. Use **super** keyword to pass parameters.
3. Display the complete information of the student.

Q3. Hierarchical Inheritance – Payroll System with Dynamic Binding

Create a base class **Employee** with:

- **name**
- **basicSalary**
- method **calculateSalary()**

Create subclasses:

- **Developer**
- **Manager**
- **Intern**

Each subclass calculates the final salary differently.

Requirements:

1. Create an **array of Employee references**.
2. Store objects of different subclasses.
3. Invoke **calculateSalary()** using the base reference to demonstrate **dynamic binding**.

Q4. Abstract Class – Banking System

Create an **abstract class BankAccount** containing:

- **accountNumber**
- **balance**

Methods:

- constructor
- abstract method **computeInterest()**
- method **displayAccountDetails()**

Create subclasses:

- `SavingsAccount`
- `FixedDepositAccount`

Requirements:

1. Use **constructor chaining**.
2. Store objects of both subclasses in a **BankAccount** reference.
3. Demonstrate **dynamic binding** when calling `computeInterest()`.

Q5. Final Class – Utility Class

Create a **final class** `MathUtility` containing methods:

- `square(int n)`
- `cube(int n)`

Requirements:

1. Write a program that calls these methods.
2. Explain why creating a subclass of `MathUtility` is not permitted.

BATCH C

Q1. Single Inheritance – Library System

Design a base class **Book** containing:

- title
- author
- isbn

Create a subclass **EBook** containing: **fileSize, format**

Requirements:

1. Implement **constructor chaining using super()**.
2. Display both book and ebook details.
3. Demonstrate the initialization sequence.

Q2. Multilevel Inheritance – Transport System

Create the following hierarchy: **Transport** → **PublicTransport** → **Bus**

Attributes:

- **Transport** → transportType
- **PublicTransport** → routeNumber
- **Bus** → seatingCapacity

Requirements:

1. Use **constructor chaining across all classes**.
2. Display messages showing constructor execution order.

Q3. Hierarchical Inheritance – Shape Cost Calculator

Create a base class **Shape** with method **calculatePaintingCost()**.

Create subclasses:

- **Square**
- **Circle**
- **Triangle**

Each calculates cost based on area and cost-per-unit.

Requirements:

1. Use **dynamic binding** through a **Shape** reference variable.
2. Demonstrate cost calculation for multiple objects.

Q4. Abstract Class – Media Player

Create an **abstract class** **Media** with:

- attribute **title**
- abstract method **play()**

Create subclasses: **Audio**, **Video**

Requirements:

1. Use **constructor chaining**.
2. Demonstrate **runtime polymorphism** using a base class reference.

Q5. Final Class – Configuration Settings

Create a **final class** **SystemConfig** containing:

- **configurationId**
- **systemName**

Add a method **displayConfiguration()**.

Requirements:

1. Create objects of this class.
2. Demonstrate why inheritance is not allowed.

BATCH D

Q1. Single Inheritance – Banking Transaction

Create a class **Account** containing:

- accountNumber
- accountHolder

Create a subclass **SavingsAccount** containing:

- interestRate

Requirements:

1. Use **super()** for constructor invocation.
2. Display the complete account details.

Q2. Multilevel Inheritance – Device Hierarchy

Create: **Device** → **Computer** → **Laptop**

Attributes: deviceName, processor, batteryLife

Requirements:

1. Implement **constructor chaining**.
2. Display initialization order using print statements.

Q3. Hierarchical Inheritance – Course Management

Create a base class **Course**.

Create subclasses:

- **OnlineCourse**
- **OfflineCourse**

Each should implement method **calculateCourseFee()** differently.

Requirements:

1. Use a **Course reference** to demonstrate **dynamic binding**.

Q4. Abstract Class – Payment System

Create an abstract class **Payment**.

Subclasses:

- **CreditCardPayment**
- **UPIPayment**

Each implements the method **processPayment()**.

Requirements:

1. Use **constructor chaining with super**.
2. Demonstrate **runtime polymorphism** using base class references.

Q5. Final Class – Logger System

Create a **final class Logger** containing method: **logMessage(String message)**

Requirements:

1. Write a program that logs different system messages.
2. Explain why the logger class should not be extended.